

Gemma Tounsi 1.0 27B

Enterprise ML Project Plan

Tunisian-adapted Gemma 3 27B via QLoRA
Training orchestration with Soup

Field	Value
Base model	Google Gemma 3 27B (instruction-tuned)
Adaptation method	QLoRA 4-bit supervised fine-tuning (SFT)
Training engine	Soup (external open-source dependency)
Budget ceiling	\$50 USD (constraint, not target)
Document version	1.0
Repository	github.com/medbensaad26-cell/Gemma-Tounsi-1.0-27B
Status	Planning & repository scaffolding

Confidential project planning document for reproducible model adaptation.

Table of Contents

- 01 Executive Summary
- 02 Project Objective
- 03 Training Strategy
- 04 Tunisian Language Strategy
- 05 Dataset Strategy
- 06 Capability Preservation Strategy
- 07 Evaluation Strategy
- 08 Training Workflow
- 09 Budget Strategy
- 10 Success Criteria
- 11 Deliverables
- 12 Roadmap

01 — Executive Summary

Motivation. Arabic NLP tools rarely account for Tunisian Darija in any of its written forms. Developers, researchers, and Tunisian users need models that understand and produce natural communication in Derja, Arabizi, and Franco-Tunisian — not just Modern Standard Arabic or generic Maghrebi approximations.

Gemma Tounsi 1.0 27B addresses this gap by adapting Google's Gemma 3 27B to understand and generate natural Tunisian communication across Arabizi, Arabic-script Derja, and Franco-Tunisian code-switching — while preserving the base model's general reasoning, coding, and mathematical capabilities.

This document defines the enterprise-grade plan for v1.0: data strategy, training approach, evaluation gates, budget constraints, and release criteria. Training is delegated to the open-source Soup engine; this project owns datasets, evaluation, reproducibility, and release orchestration.

Key principle: adaptation, not from-scratch training. We teach an existing capable model a new linguistic skill through targeted QLoRA fine-tuning under strict quality and budget controls.

02 — Project Objective

Primary objective: adapt Gemma 3 27B to understand and communicate naturally in Tunisian across all major written forms, without degrading general capabilities.

02.1 — Adaptation, Not From-Scratch Training

Gemma Tounsi 1.0 27B is not a new foundation model. The base Gemma 3 27B already provides strong instruction-following, reasoning, coding, and multilingual ability. This project adds a targeted linguistic adaptation layer through supervised fine-tuning with QLoRA — training a small adapter, not rewriting 27 billion parameters.

02.2 — What Success Looks Like

Success is achieved when the adapted model (1) responds naturally and accurately in Tunisian when prompted in Derja, Arabizi, or Franco-Tunisian; (2) generates output in the user's chosen script and register rather than drifting to MSA; (3) follows instructions reliably in dialectal contexts; and (4) does not regress on math, coding, or general reasoning benchmarks beyond the thresholds defined in Section 10.

02.3 — Explicit Non-Goals for v1.0

- Continued pretraining or full-model fine-tuning
- Multi-stage DPO/RLHF pipelines within the \$50 budget
- Model serving infrastructure (deferred to downstream consumers)
- Merged full-weight publication (adapter-only release on Hugging Face)

03 — Training Strategy

v1.0 uses a single QLoRA SFT run with explicit replay data for capability preservation. Hyperparameters below are planning targets; final values are confirmed only after Soup environment

validation and soup profile on real hardware.

03.1 — QLoRA Hyperparameter Targets

Parameter	Target	Notes
Base model	google/gemma-3-27b-it	Pinned revision before training
Quantization	4-bit NF4 (QLoRA)	Standard for 24 GB GPU training
LoRA rank (r)	32	Balance capacity vs. overfitting
LoRA alpha	64	Typically 2× rank
Target modules	q_proj, k_proj, v_proj, o_proj	Attention projections
Sequence length	512–768 tokens	Confirm via soup profile
Epochs	1	Pilot may use subset; avoid over-training
Batch strategy	Micro-batch 1 + gradient accumulation	Auto-tuned by Soup where possible

03.2 — Token-Weighted Training Mixture

The initial draft used a 40/30/15/15 split. The revised v1.0 plan adds an explicit 20% capability-retention slice. MSA exposure alone does not protect math, coding, or reasoning.

Category	% Tokens	Rationale
Tunisian Arabizi	35%	Highest orthographic variation; primary adaptation target
Arabic-script Derja	25%	Core dialect coverage in native script
Franco-Tunisian	12%	Common online register; natural code-switching
MSA / Formal Arabic	8%	Register coverage; not a substitute for retention
Capability retention	20%	Explicit replay: math, code, reasoning, general instructions

Cross-cutting rule: at least 20–25% of examples within the Arabizi and Derja slices must involve reasoning, math, or coding — not conversational chit-chat alone.

04 — Tunisian Language Strategy

Tunisian is not a single written form. Gemma Tounsi must handle the full spectrum of real-world Tunisian communication, including orthographic variation and code-switching.

04.1 — Supported Written Forms

- Arabizi — Latin-script Tunisian (e.g., 3lech, 9adeh, nheb)
- Derja — Tunisian Arabic in Arabic script
- Franco-Tunisian — Mixed French/Tunisian register common online

04.2 — Orthographic Variation (Arabizi)

Arabizi has no standard spelling. The model must tolerate and correctly interpret multiple valid spellings of the same word or phrase.

Meaning	Example spellings
How much?	Chnowa / Chneya / Chnoua / Ch7al
Why?	3lech / Alech / Alach
I want	Nheb / N7eb / Ne7eb

04.3 — Comprehension and Generation

Adaptation requires both understanding and generation. The model must respond in the form the user chose: Arabizi input → natural Arabizi output; Derja input → natural Derja output. Drifting to MSA or generic Arabic when the user writes in dialect is a failure mode this project explicitly targets.

05 — Dataset Strategy

Data quality and provenance drive adaptation success more than hyperparameter tuning. The pipeline follows enterprise practice: collect, clean, deduplicate, validate, version, and split — with evaluation data strictly held out.

05.1 — Data Pipeline Stages

Stage	Purpose	Soup tooling
Inspect	Schema, counts, language mix	soup data inspect
Validate	Format and quality checks	soup data validate
Deduplicate	Remove exact/near duplicates	soup data dedup
Score	Quality heuristics / filtering	soup data score
Split	Deterministic train/eval separation	Manifest-driven scripts

05.2 — Target Dataset Size

Target range: 20,000–50,000 training examples for the v1.0 run. The pilot uses a representative subset (approximately 1,000–2,000 examples) to validate the pipeline before committing the full budget.

05.3 — Provenance and Licensing

Every corpus is declared in data/manifests/ with source, license, revision, and permitted use before entering any mixture. Undeclared data must never reach training files. Large artifacts remain outside Git; manifests and preparation scripts provide reproducibility.

05.4 — Contamination Control

Evaluation sets (TounsiBench, Arabizi benchmark, retention holdout) are evaluation-only. Any overlap between training and evaluation data is a pipeline failure — not something to filter silently after the fact.

06 — Capability Preservation Strategy

Dialect adaptation without explicit retention data commonly produces fluent but degraded models — strong on chat, weak on math and code. Gemma Tounsi addresses this with a dedicated 20% retention slice and Tunisian capability examples.

06.1 — Retention Data (20% of Mixture)

The retention slice includes general instruction-following, mathematical reasoning, coding, and broad knowledge tasks — drawn from established capability datasets and replay corpora. This is distinct from the 8% MSA slice, which covers register but not structured reasoning.

06.2 — Tunisian Capability Examples

Within Tunisian slices, at least 20–25% of examples involve math, reasoning, or coding prompts written in Arabizi or Derja. Source material (GSM8K, HumanEval, MMLU, instruction datasets) is rewritten so the prompt is Tunisian while the answer remains correct.

06.3 — Replay During Training

Soup's replay mechanism (soup train --replay) interleaves retention examples during training so the model continues to practice general capabilities alongside dialect adaptation.

07 — Evaluation Strategy

Evaluation is split into three independent tracks. Scores are never aggregated into a single headline number — a dialect gain that costs general capability must remain visible.

07.1 — Evaluation Tracks

Track	Measures	Directory
TounsiBench	Derja in Arabic script	eval/tounsibench/
Arabizi	Latin-script Tunisian + code-switching	eval/arabizi/
Retention	General capability regression guard	eval/retention/

07.2 — Automated Evaluation (Soup)

- soup eval — run benchmark suites against a checkpoint
- soup diagnose — analyze failure modes and regressions
- soup ship — release gate comparing candidate vs. baseline

07.3 — Human Evaluation Rubric

Human evaluation covers criteria that automatic metrics miss: naturalness, fluency, dialect authenticity, instruction following, and appropriate code-switching.

Criterion	Scale	Protocol
Naturalness	1–5	Minimum 3 native Tunisian speakers
Fluency	1–5	Independent ratings; report mean + agreement
Dialect authenticity	1–5	Tunisian-specific, not generic Maghrebi
Instruction following	1–5	Task completion in dialectal context

Baseline requirement: every reported score is paired with the same measurement on unmodified base Gemma 3 27B at pinned revision, using identical prompts and decoding settings.

08 — Training Workflow

The workflow follows a pilot-first approach: validate environment, data, and settings on a small run before committing the full training budget.

Stage	Activity	Soup / project commands
1. Environment	Validate GPU, Soup install, memory estimate	soup doctor, soup profile, soup cost
2. Data preparation	Build train.jsonl and retention.jsonl	soup data inspect, dedup, validate; scripts/prepare_data.sh
3. Baseline eval	Score unmodified Gemma 3 27B on all tracks	soup eval benchmark; scripts/evaluate.sh
4. Pilot training	Small subset run; verify loss and samples	soup train (subset config)
5. Final training	Full QLoRA SFT with replay	soup train --replay
6. Evaluation gate	All three tracks + retention threshold	soup ship; scripts/evaluate.sh
7. Release	Publish adapter + model card	soup push, soup card; scripts/release.sh

Compute model: data preparation on local PC; training on rented cloud GPU inside Docker; evaluation and Hugging Face upload from PC after downloading the adapter.

09 — Budget Strategy

Budget is a constraint, not a target. The goal is to achieve v1.0 quality within \$50 USD — not to spend \$50 by default. Pilot-first workflow minimizes wasted GPU hours.

Item	Estimated duration	Estimated cost (USD)
Environment setup + model download	1–2 hours	\$1–3
Pilot run (RTX 4090 class)	2–3 hours	\$1–2
Final training run	8–12 hours	\$5–10
Evaluation + diagnostic re-runs	2–4 hours	\$3–5
Buffer for failed attempts / tuning	—	\$5
Transfer, storage, misc	—	\$3
Total ceiling	—	≤ \$50

GPU selection is deferred until soup profile reports actual VRAM requirements. Target hardware: 24 GB VRAM (RTX 4090 / RTX 3090 class). Upgrade only if profiling demands it.

10 — Success Criteria

Success criteria are measurable. Thresholds below are planning targets for v1.0; final values may be adjusted before the first baseline run, but never post-hoc to justify a failed model.

#	Criterion	Threshold
1	TounsiBench improvement vs. baseline	≥ 20% relative improvement on primary metric
2	Arabizi benchmark improvement vs. baseline	≥ 20% relative improvement on primary metric
3	Human eval (naturalness + authenticity)	Mean score ≥ 4.0 / 5.0 across raters
4	Instruction following in dialect	≥ 85% task completion on held-out Tunisian tasks
5	Retention regression vs. baseline	No track drops more than 5% relative to baseline
6	Contamination check	Zero eval items found in training mixtures
7	Reproducibility	Full run reproducible from repo commit + frozen config

11 — Deliverables

v1.0 release produces the following artifacts:

- QLoRA adapter weights published on Hugging Face Hub
- Hugging Face model repository (e.g., you/Gemma-Tounsi-1.0-27B) with model card
- Frozen release configuration (configs/releases/v1.0.yaml)
- Evaluation report covering all three tracks with baseline comparisons

- Dataset manifests with full provenance and licensing records
- Reproducibility record: repo commit, Soup SHA, seeds, GPU info, checksums
- This repository as the source-of-truth for data, eval, and release orchestration

12 — Roadmap

Phase	Activities	Est. duration
Phase 0 — Scaffolding	Repository architecture, manifests, documentation	Complete
Phase 1 — Environment	Soup validation, Docker, soup doctor/profile	~2–3 days
Phase 2 — Data	Source identification, licensing, preparation pipeline	~1–2 weeks
Phase 3 — Baseline	Define benchmarks, run base model eval	~3–5 days
Phase 4 — Pilot	Subset training, sample review, config tuning	~2–3 days
Phase 5 — Final train	Full QLoRA run with replay	~1–2 days (GPU time)
Phase 6 — Eval & release	All tracks, ship gate, HF publish	~3–5 days

End state: a reproducible, publicly documented Tunisian adaptation of Gemma 3 27B that downstream applications (RAG systems, chat interfaces, research tools) can consume via the Hugging Face adapter — without this repository owning product serving infrastructure.